ONE-GO AI STUDIO PROMPT

Panabo City Hall Civil Registry Queue Management System

Detailed application specification with Firebase, Supabase, QR registration, and PWD priority

---

Purpose: Build a functional digital queue system for citizens requesting birth, marriage, and death certificates in the Civil Registry service context of Panabo City Hall.

Prompt structure: Context, Objective, Requirements, Constraints, Output.

How to use this PDF

- Open Google AI Studio and start a new app/build session.
- Copy the full prompt in Sections 1–5 into AI Studio. If its prompt limit requires multiple messages, submit each section in order and tell it to preserve the earlier requirements.
- Connect Firebase and Supabase using your own projects and credentials. Never place private service-role keys in browser code.
- After generation, run the acceptance checklist at the end. Do not assume a feature works just because AI Studio generated a button for it.

Scope decision: The original activity specifies one logical queue. Request categories are visible to staff for filtering, but all categories share one authoritative queue and sequence. This avoids silently introducing multiple independent queues. Separate quick-service/drop-off lanes should be added only if the instructor or office explicitly changes the scope.

THE ONE-GO PROMPT

Copy everything between START OF PROMPT and END OF PROMPT into Google AI Studio.

START OF PROMPT

You are a senior full-stack engineer, product designer, database architect, security engineer, and QA specialist. Build a complete, working, responsive web application called **Panabo City Hall Civil Registry Queue Management System**. Do not stop at a mockup or static dashboard. Implement the front end, backend integrations, database structure, access controls, real-time queue updates, validation, loading/error states, and testable end-to-end workflows. If an external service requires setup or credentials you cannot configure, implement the integration and provide precise setup instructions. Never pretend an integration is connected unless verified.

1. CONTEXT

The system is for the Civil Registry service context of Panabo City Hall, where citizens request birth, marriage, and death certificates. Manual lines or paper numbering can create confusion about who is next, make queue status difficult to see, and require staff to coordinate customer turns manually. Citizens may also have difficulty standing for long periods. Staff need a consistent way to track Waiting, Serving, Served, and Skipped requests.

The application improves organization, clarity, accessibility, privacy, and accountability. It is an educational queue-management prototype, not a replacement for official civil registry records or certificate issuance. Do not collect sensitive certificate details, government identification numbers, dates of birth, document scans, or details about the person named on a certificate. Collect only the minimum information required for queue registration.

The required scope is one logical service queue. Citizens select Birth Certificate, Marriage Certificate, or Death Certificate as request metadata. All categories share one authoritative queue order and one sequence of queue numbers. Do not create multiple independent queues, counters, appointments, SMS/email notifications, waiting-time predictions, certificate applications, payments, or advanced analytics unless explicitly requested later.

Include QR-based queue joining and a PWD priority option. Also provide staff-assisted registration for citizens without a smartphone or who need assistance. Support phones, tablets, laptops, and a large public display.

2. OBJECTIVE

Deliver a polished, reliable, accessible web application that lets citizens join the Civil Registry queue, receive a unique queue ticket, and check their status. Authorized staff must call the next eligible person and manage each ticket's lifecycle. A separate public display shows the current serving number and queue information in real time without exposing full names or sensitive information.

Success means a citizen can register once and receive a unique number; staff can call the next eligible ticket; all screens stay consistent; Served and Skipped tickets cannot be called again; priority behavior is deterministic and auditable; and unauthorized users cannot modify queue records.

3. REQUIREMENTS

3.1 User roles and permissions

- **Citizen:** Can register a ticket, select request category, optionally request PWD priority, see their ticket number/status, and view public queue information. Cannot access staff controls or edit another person's ticket.
- **Staff:** Must sign in. Can view the waiting list, call the next eligible customer, mark a ticket Serving, Served, or Skipped, register walk-ins, and verify/decline priority if authorized.
- **Administrator:** Can manage authorized staff access, view audit history, and close/reset sessions. Admin actions must be protected and logged.
- **Public display:** Read-only. Must not expose staff controls, full names, phone numbers, private ticket data, or audit logs.

Enforce permissions in Firebase/Firestore rules and Supabase policies or trusted server-side code. Hiding a button is not sufficient security.

3.2 Citizen registration flow

1. Provide a landing page clearly identifying the Panabo City Hall Civil Registry Queue Management System. State that this is queue registration only, not online certificate application or delivery.
2. Provide a Join the Queue action and a QR route. The QR encodes the actual registration URL. Document how to configure the deployed URL; never use a fabricated production URL or localhost in the deployed QR.
3. Ask for the citizen's display name in one field. Trim whitespace, reject blank input, limit length, and safely render user-entered text. Do not ask for unnecessary personal data.
4. Require a request category: Birth Certificate, Marriage Certificate, or Death Certificate. Category selection is metadata and does not create a separate queue.
5. Offer an optional PWD priority request with respectful wording. Do not ask for diagnosis or medical details. Store priority_requested separately from priority_verified.
6. Assign a unique queue number through trusted backend logic using an atomic operation or transaction. Never let the browser decide the next number or accept a client-supplied number.
7. Show a ticket confirmation with queue number, category, current status, and instructions to watch the public display. Do not promise an exact waiting time.
8. Prevent accidental duplicate submissions with a loading state and safe retry/idempotency. Do not block legitimate separate requests without a clear policy.
9. On failure, show an understandable message and allow retry without creating duplicate tickets.

3.3 Queue number and lifecycle rules

- Use one queue and one authoritative sequence for all three request categories. A readable format such as Q-001 is acceptable. Store session identity separately so the same display number can be reused only in a new session.
- Generate numbers atomically in trusted backend code. Simultaneous registrations must never receive the same number. Never derive the number from the browser's waiting-list length.
- Each ticket has a stable internal ID, session ID, queue number, display name, request category, priority-request/verification fields, status, created/updated timestamps, and relevant called/served/skipped timestamps.
- Allowed statuses are Waiting, Serving, Served, and Skipped. Use consistent enums throughout the application.
- Only one ticket can be Serving at a time in this single queue. Enforce this through trusted logic.
- Call Next selects the next eligible Waiting ticket according to the configured priority and fairness rules. Exclude all Served and Skipped tickets. Do not call a second ticket while another remains Serving unless staff first completes or skips the current one.
- Once marked Served or Skipped, a ticket is permanently ineligible for Call Next through normal staff controls. Do not allow invalid transitions such as Served back to Waiting.
- Preserve skipped tickets and their audit records. Do not silently delete or automatically reinsert them.
- Confirm consequential actions such as Served, Skip, and session reset. Prevent double-clicks and repeated backend requests.
- Provide empty states and safe behavior when nobody is waiting or the queue is exhausted.

3.4 PWD priority behavior

Use a transparent, configurable policy. Default behavior: a PWD-priority ticket verified by authorized staff goes ahead of ordinary Waiting tickets while preserving first-come, first-served order within each group. A priority ticket must not interrupt a ticket already Serving. A self-declared priority request does not receive verified priority unless the administrator explicitly configures that policy. Document that the actual ordering must be approved by the intended office.

- Keep priority_requested and priority_verified as separate fields.
- Only authorized staff can verify or decline a priority request. Record who changed it and when.
- Do not require uploading a PWD ID or storing medical details for this prototype. If actual office policy requires proof, staff may verify it in person without saving a copy.
- Show a restrained PWD Priority indicator on the staff screen only. The public display must not reveal a person's name or disability status.
- Make the priority policy easy to configure so the office can use its approved rules.

3.5 Staff dashboard

- Require staff authentication and provide sign-out.
- Prominently show the current Serving ticket, queue number, and request category. Display the customer's name only on the protected staff screen when operationally needed.
- Show a waiting list with queue number, category, registration time, status, and priority verification. Provide category and priority filters, but filters must not change actual queue order.
- Provide clear actions: Call Next, Mark Serving if a separate step is used, Mark Served, Skip, and verify/decline priority where permitted.
- If Call Next automatically moves a ticket to Serving, make that behavior explicit and keep the lifecycle consistent.
- Show basic counts for Waiting, Serving, Served, and Skipped. Avoid unnecessary analytics or decorative charts.
- Include loading, empty, error, and retry states. Prevent double-clicks and show a connection indicator only if it accurately reflects connection state.
- Allow protected staff search by queue number or display name if feasible. Search does not replace authorization.

3.6 Public queue display

- Create a dedicated read-only route optimized for a television or wide monitor, with large typography, high contrast, and minimal clutter.
- Show NOW SERVING with the current queue number as the most prominent element. Show a small, readable subset of upcoming queue numbers and total waiting count.
- Do not display full names, phone numbers, PWD status, or private information. Show request category only if useful and readable.
- Update in real time without manual refresh. If live updates fail, show a subtle stale-data/connection notice and recover when connectivity returns.
- Sound is optional, must be muteable, and must never be the only way to communicate a turn.

3.7 QR-based joining

- Generate a QR code that encodes the registration page URL, never personal data or a private ticket URL.
- Provide a staff/admin screen or setup instructions to view and print the QR code. It must use the deployed URL after deployment, not localhost.
- Scanning opens the registration page in a normal mobile browser; no app installation is required.
- QR registration uses the same server validation, queue-number allocation, priority rules, and abuse protections as staff-assisted registration.

3.8 UI and UX design

- Use a calm, trustworthy public-service visual style suitable for a Philippine local-government service. Keep it professional, not overly decorative. Do not use official seals/logos unless provided; use replaceable neutral branding.
- Use a consistent design system for spacing, typography, buttons, forms, status badges, cards, dialogs, and responsive breakpoints.
- Build separate responsive views for citizen registration, ticket confirmation/status, staff sign-in, staff dashboard, public display, and session administration.
- Use plain English, short instructions, obvious primary actions, and as few steps as practical to join.
- Follow accessibility best practices: semantic headings, labels, keyboard navigation, visible focus, sufficient contrast, scalable text, readable errors, and comfortable touch targets. Do not communicate status by color alone.
- Use only minimal animations: short fade/slide transitions, button feedback, small loading indicators, and a subtle emphasis when the serving number changes. Avoid distracting bouncing, constant pulsing, parallax, animated backgrounds, or motion that delays actions.

- Respect reduced-motion preferences where supported. Keep the site usable on modest phones and office computers, with no unnecessarily heavy assets.
- Clearly label any seeded sample data as development/demo data. Never mix it silently with live queue records.

3.9 Backend architecture: Firebase and Supabase

Use both Firebase and Supabase with clearly separated responsibilities. Do not duplicate every operational record in both databases or create conflicting sources of truth. Use Firebase for the live operational queue and Supabase for structured audit/history records unless a documented technical constraint requires a different design.

Firebase

- **Firebase Authentication:** Staff identity and sign-in. Citizen registration does not require a citizen account in this prototype, but must use trusted validation and reasonable abuse protection.
- **Cloud Firestore:** Active session metadata, queue tickets, status, queue number, request category, priority fields, and current serving state.
- **Trusted operations:** Implement number allocation, Call Next, status transitions, and priority ordering in a trusted backend such as Cloud Functions or a server API. Use transactions/atomic operations.
- **Firestore rules:** Deny unrestricted public writes. Public clients read only the minimum public display projection or submit registration through a validated endpoint. Protect staff actions and document/test rules.
- **Role assignment:** Use trusted role claims or a protected role mapping. Users must not be able to grant themselves staff/admin access by editing a profile. Never hardcode admin credentials.

Supabase

- **PostgreSQL:** Store structured audit events and finalized session history where appropriate. Define schema, keys, timestamps, indexes, and relationships.
- **RLS:** Enable Row Level Security on exposed tables and write explicit policies. A publishable/anon key is not a substitute for correct RLS.
- **Trusted writes:** Keep the Supabase service-role key only in server-side environment secrets. Never put it in browser code, public configuration, Git, or screenshots.
- **Consistency:** Define how Firestore operational events reach Supabase. Use event IDs/idempotency keys to prevent duplicate audit records. Handle failed writes with retry or a documented fallback; do not claim cross-database transactions are atomic.
- **Environment:** Use environment variables for project URLs and client-safe keys. Provide .env.example with placeholder names only and exclude .env from Git.

> If the AI Studio environment cannot deploy a secure trusted backend automatically, do not fake server-side security in client code. Generate the required server functions/API and precise setup steps, and label any external setup that remains incomplete.

3.10 Suggested data model

- **QueueSession (Firestore):** session_id, service_name, date, status (open/closed), next_number/counter, current_serving_ticket_id, created_at, updated_at. Protect counter changes with trusted transactions.
- **QueueTicket (Firestore):** ticket_id, session_id, queue_number, display_number, display_name, request_category (birth/marriage/death), priority_requested, priority_verified, status (waiting/serving/served/skipped), created_at, updated_at, called_at, served_at, skipped_at, registration_source.
- **Staff role mapping:** Firebase UID and trusted role information. Role assignment is restricted to authorized admin/server processes.
- **AuditEvent (Supabase):** event_id, session_id, ticket_id where relevant, actor identifier, action_type, previous_status, new_status, event_timestamp, request_id/idempotency_key, and minimal non-sensitive metadata. Avoid unnecessary names in logs.
- **QueueSessionHistory (Supabase, optional):** session_id, session date, total registered, served, skipped, and timestamps. Do not build advanced analytics.
- Use server timestamps, validate enums and allowed transitions, and add indexes for session/status/queue order and waiting-list queries.

3.11 Session opening, closing, and reset

- Support one active queue session at a time. Show the date and session status to staff.
- Provide an admin-only close/reset action with confirmation and a summary of remaining Waiting/Serving tickets.
- Never silently delete old tickets on reset. Preserve required audit/history and mark the old session closed.
- Start a new numbering sequence only in a new session. Display numbers may restart at Q-001, but internal ticket and session IDs remain unique.
- Prevent public registration when the queue is closed and show a helpful message. Staff can intentionally close registration.

3.12 Error handling and reliability

- Handle network loss, permission errors, expired staff sessions, invalid inputs, duplicate submissions, failed number allocation, failed audit writes, and closed sessions.
- Never show success until the trusted backend confirms ticket creation.
- Use idempotency protection so retries do not create duplicate tickets or repeat state transitions.
- Show understandable citizen-facing errors and useful diagnostics only to authorized developers/admins. Do not expose secrets.
- Log safely: never log tokens, passwords, service-role keys, certificate details, or unnecessary personal data.
- Document what happens if Firestore is unavailable or Supabase audit storage fails. Never silently claim full success when a required operation failed.

3.13 Security and privacy

- Validate input on client and server; safely render user-supplied names.
- Use least privilege. Public users must not read staff records, private ticket details, audit logs, or internal session settings.
- Keep secrets in environment variables or a hosting secret manager. Never embed private keys in front-end code.
- Enable and test Firestore rules and Supabase RLS with deny-by-default behavior.
- Use HTTPS in deployment; do not describe a local development server as production-ready.
- Do not collect certificate application details, government ID numbers, document images, contact details, or medical details.
- Public screens expose only the minimum data necessary to identify a turn. Do not publicly show names or PWD status.
- Audit queue status changes, priority verification, session closure, resets, and admin actions.
- Do not implement staff impersonation or a production login bypass. Any demo mode must be explicitly development-only.

3.14 Testing and acceptance criteria

- **Registration:** valid citizen can join; blank/too-long names are rejected; category is required; repeated click does not duplicate tickets.
- **Unique numbers:** simultaneous registrations receive distinct numbers; test concurrency or document the test procedure.
- **Queue order:** Call Next follows the configured priority rule and preserves order within each group.
- **Served/skipped protection:** Served and Skipped tickets are never called again. Invalid transitions are rejected by backend logic.
- **Single Serving:** system prevents two tickets becoming Serving at once.
- **PWD priority:** unverified self-declaration does not receive verified priority unless configured; verification is role-protected and audited.
- **Live updates:** registration and status changes appear on staff and public screens without manual refresh.
- **Permissions:** unauthenticated users cannot perform staff actions; staff cannot perform admin-only actions; public clients cannot read audit data.
- **QR:** QR opens the correct deployed registration page on a phone, not localhost.
- **Privacy:** public display does not reveal names, PWD status, phone numbers, or private information.

- **Failures:** test network loss, backend denial, failed audit write, and closed session; UI must avoid false success.
- **Accessibility/responsiveness:** test mobile/tablet/desktop/large display, keyboard focus, labels, contrast, and reduced motion.
- **Reset:** only admin can close/reset; history remains preserved and new numbers belong to a new session.

4. CONSTRAINTS

- Keep one logical service queue, one active Serving ticket, and one authoritative sequence. Request categories are labels/filters, not separate queues.
- Do not implement multiple counters, appointments, SMS/email, wait-time prediction, certificate application submission, document upload, online payments, certificate generation, QR identity verification, or advanced analytics.
- QR is a link to the registration page, not an identity credential.
- PWD priority is configurable and must not be represented as confirmed official legal policy. Explain that the office must approve the rule.
- Keep the UI clean, accessible, fast, and appropriate for a public-service environment, with minimal animations.
- Do not use fake backend calls, hardcoded live queue data, browser-only counters, or mock success messages instead of real integration.
- Do not invent project IDs, credentials, staff accounts, official logos, official URLs, or deployment results.
- Never expose secrets in client code or documentation. Do not commit .env or credentials.
- If external setup is required, state what is implemented and what remains. Do not claim deployment, authentication, database connection, or live sync succeeded unless verified.
- Use maintainable code, reusable components, typed models where supported, centralized validation, and comments for critical queue rules.

5. OUTPUT

Deliver the following in the AI Studio project:

- **Complete responsive app:** citizen registration, ticket confirmation/status, staff sign-in, staff dashboard, public display, QR flow, and admin-only session management.
- **Firebase:** Authentication, Firestore operational queue records, trusted number allocation and status transitions, live updates, security rules, and required indexes.
- **Supabase:** PostgreSQL schema/migrations for audit/history, RLS policies, safe server-side writes, and documented consistency/retry behavior.
- **Environment template:** .env.example with placeholders only, .gitignore excluding secrets, and safe configuration instructions.
- **Database setup:** Firestore rules/indexes and reproducible Supabase SQL migrations or equivalent setup scripts.
- **README:** prerequisites, project structure, local setup, Firebase setup, Supabase setup, staff role assignment, environment configuration, run/deploy steps, QR URL configuration, security notes, and limitations.
- **Tests:** automated tests where practical and a manual acceptance checklist.
- **Demo data:** optional, clearly labeled development-only seed data.
- **Completion report:** implemented features, files created/changed, external setup remaining, tests run and results, and known issues. Separate verified behavior from assumptions.

Implementation workflow

1. Inspect the available AI Studio project and choose a supported stack. Briefly state the framework, trusted backend approach, and Firebase/Supabase responsibility split.
2. Create the data model, lifecycle rules, and security model before connecting the UI.
3. Build the responsive interface with reusable components and realistic loading, empty, error, confirmation, and success states.

4. Implement trusted queue operations and authorization. Test unique number allocation, priority ordering, and invalid-transition protection.
5. Connect live data to staff and public screens. Add Supabase audit/history with idempotency and error handling.
6. Add QR registration, accessibility refinements, and minimal motion.
7. Run tests, fix failures, and include backend security tests.
8. Provide setup instructions and a completion report. Ask for information only when genuinely required, such as project credentials or deployment URL.

Final instruction: Prioritize correctness, privacy, reliable queue ordering, and complete end-to-end functionality over decoration. If a requirement cannot be implemented in this environment, explain the limitation and provide secure implementation/setup steps. Never simulate a successful external integration.

END OF PROMPT

POST-GENERATION ACCEPTANCE CHECKLIST

Use this checklist after AI Studio generates the app. Verify the behavior yourself; do not rely only on the AI's summary.

Core workflow

- Citizen can register a name and choose Birth, Marriage, or Death Certificate.
- A unique queue number is assigned by trusted backend logic.
- Citizen sees ticket confirmation and current status.
- QR code opens registration in a mobile browser.
- Staff can register walk-ins for citizens needing assistance.
- Staff sign-in is required for queue controls.
- Call Next follows the configured priority rule and never calls Served/Skipped tickets.
- The single queue never has more than one Serving ticket.
- Status changes update staff and public display views in real time.
- Public display does not show full names or PWD status.

Backend and security

- Firebase authentication and real database reads/writes are tested.
- Firestore rules deny unauthorized public writes and private data reads.
- Number allocation and status transitions run in trusted backend code.
- Supabase tables and RLS policies are applied and tested.
- Service-role credentials are server-side only and absent from client bundles/Git.
- Audit writes are idempotent and failures are handled explicitly.
- .env is excluded from version control; .env.example contains placeholders only.
- Staff and admin permissions are tested separately.

User experience

- Registration works on a phone without horizontal scrolling.
- Inputs have labels, keyboard focus is visible, and errors explain recovery.
- Animations are brief and do not delay actions.
- Public display is readable at a distance and indicates stale connection if appropriate.
- QR points to the correct deployed URL, not localhost.

Suggested demo sequence

1. Register a Birth Certificate request from a phone-sized view.
2. Register Marriage and Death requests to demonstrate the shared queue.
3. Request PWD priority for one ticket, then verify it as authorized staff.
4. Open staff dashboard and public display in separate tabs.
5. Call the next eligible ticket and verify both views update.
6. Mark the ticket Served and verify it cannot be called again.
7. Skip another ticket and verify it cannot be called again.
8. Attempt a staff action while signed out and verify the backend denies it.

9. Demonstrate session reset only with an admin role and confirm that old history remains.

> This is a queue-management prototype, not an official certificate-request portal. Registering a queue ticket does not submit a certificate application or guarantee service. Confirm actual PWD priority and quick-service/drop-off policy with the instructor or intended office. If the assignment explicitly changes to multiple lanes, update queue architecture and fairness rules deliberately.

End of document.